

Selectors: Identifying What to Style

Every CSS style has two basic parts: a selector and a declaration block. (And if that's news to you, go back and read the previous chapter.) The declaration block carries the formatting properties—text color, font size, and so on—but that's just the pretty stuff. The ability to focus your styling on specific items lies in those first few characters at the beginning of every rule—the selector. By telling CSS *what* you want it to format (see Figure 3-1), the selector gives you full control of your page's appearance. If you're into sweeping generalizations, then you can use a selector that applies to many elements on a page at once. But if you're a little more detail oriented (OK, a *lot* more), other selectors let you single out one specific item or a collection of similar items. CSS selectors give you a lot of power; this chapter shows you how to use them.

```
h1 {  
    font-family: Arial, sans-serif;  
    color: #CCCCFF;  
}
```

FIGURE 3-1

The first part of a style, the selector, indicates the element or elements of a page to format. In this case, h1 stands for “every heading 1, or <h1>, tag on this page.”

NOTE

If you'd rather get some hands-on experience before studying the ins and outs of CSS selectors, then jump to the tutorial on page 70.

■ Type Selectors: Styling HTML Tags

Selectors used to style particular HTML tags are called *type* or *element* selectors. They are extremely efficient styling tools, since they apply to every occurrence of that tag on a web page. With them, you can make sweeping design changes to a page with very little effort. For example, when you want to format every paragraph of text on a page, using the same font, color, and size, you merely create a style using `p` (as in the `<p>` tag) as the selector. In essence, a type selector redefines how a browser displays a particular tag.

Prior to CSS, in order to format text, you had to wrap that text in a `<font>` tag. To add the same look to every paragraph on a page, you often had to use the `<font>` tag multiple times. This process was a lot of work and required a lot of HTML, making pages slower to download and more time-consuming to update. With type selectors, you don't actually have to do anything to the HTML—just create the CSS rule, and let the browser do the rest.

Type selectors are easy to spot in a CSS rule, since they bear the exact same name as the tag they style—`p`, `h1`, `table`, `img`, and so on. For example, in Figure 3-2, the `h2` selector (top) applies some font styling to all `<h2>` tags on a web page (bottom).

```
h2 {  
  font-family:"Century Gothic", "Gill Sans", sans-serif;  
  color:#000000;  
  margin-bottom:0;  
}
```



FIGURE 3-2

A type selector affects every instance of the tag on the page. This page has three `<h2>` tags (indicated by the black labels at the left edge of the browser window). A single CSS style with a selector of `h2` controls the presentation of every `<h2>` tag on the page.

NOTE

As Figure 3-2 makes clear, type selectors don't get the less than (<) and greater than (>) symbols that surround HTML tags. So when you're writing a rule for the <p> tag, for example, just type the tag's name: p.

Type selectors have their downsides, however. What if you want *some* paragraphs to look different from other paragraphs? A simple type selector won't do, since it doesn't provide enough information for a web browser to identify the difference between the <p> tags you want to highlight in purple, bold, and large type from the <p> tags you want to leave with normal, black text. Fortunately, CSS provides several ways to solve this problem—the most straightforward method is called a *class selector*.

■ Class Selectors: Pinpoint Control

When you want to give one or more elements a look that's different from related tags on the page—for example, give one or two images on a page a red border while leaving the majority of other images unstyled—you can use a *class selector*. If you're familiar with styles in word-processing programs like Microsoft Word, then class selectors will feel familiar. You create a class selector by giving it a name and then applying it to just the HTML tags you wish to format. For example, you can create a class style named .copyright and then apply it only to a paragraph containing copyright information, without affecting any other paragraphs.

Class selectors also let you pinpoint an exact element, regardless of its tag. Say you want to format a word or two inside a paragraph, for example. In this case, you don't want the entire <p> tag affected, just a single phrase inside it. You can use a class selector to indicate just those words. You can even use a class selector to apply the same formatting to multiple elements that have different HTML tags. For example, you can give one paragraph and one second-level heading the same styling—perhaps a color and a font you've selected to highlight special information, as shown in Figure 3-3. Unlike type selectors, which limit you to the existing HTML tags on the page, you can create as many class selectors as you like and put them anywhere you want.

NOTE

When you want to apply a class selector to just a few words contained inside another tag (like the middle paragraph in Figure 3-3), you need a little help from the tag. See the box on page 47 for more detail.

You've probably noticed the period that starts every class selector's name—such as .copyright and .special. It's one of a few rules to keep in mind when naming a class:

- **All class selector names must begin with a period.** That's how web browsers spot a class selector in the style sheet.
- **CSS permits only letters, numbers, hyphens, and underscores in class names.**

```
.special {
  color:#FF0000;
  font-family:"Monotype Corsiva";
}
```



class="special"

FIGURE 3-3

Class selectors let you make highly targeted design changes. For example, you can stylize one instance of an `<h2>` heading ("Wet Sod is Heavy Sod"). The class selector `.special` tells the browser to apply the style to just that single `<h2>` tag. Once you've created a class selector, you can use it on other tags, like the top paragraph on this page.

- **After the period, the name must always start with a *letter*.** For example, `.9lives` isn't a valid class name, but `.crazy8` is. You can have classes named `.copy-right` and `.banner_image`, but not `.-bad` or `._as_bad`.
- **Class names are case-sensitive.** For example, CSS treats `.SIDEBAR` and `.sidebar` as two different classes.

Apart from the name, you create class styles exactly like tag styles. After the class name, simply slap on a declaration block containing all the styling you desire:

```
.special {
  color:#FF0000;
  font-family:"Monotype Corsiva";
}
```

Because type selectors apply across the board to all tags on a web page, you merely have to define them in your style sheet: The HTML tags that make them work are

already in place. The extra freedom you get with class styles, though, comes with a little more work. Using class selectors is a two-step process. After you create a class rule, you must then indicate *where* you want to apply that formatting. To do so, you add a class attribute to the HTML tag you wish to style.

Say you create a class `.special` that you'll use to highlight particular page elements. To add this style to a paragraph, add a class attribute to the `<p>` tag, like so:

```
<p class="special">
```

NOTE

In the HTML, as part of the class attribute, you *don't* put a period before the class name. The period is only required for the class selector name in a *style sheet*.

When a web browser encounters this tag, it knows to apply the formatting rules contained in the `.special` style to the paragraph. You can also apply class formatting to only part of a paragraph or heading by adding a `<span>` tag. For example, to highlight just one word in a paragraph using the `.special` style, you could write this:

```
<p>Welcome to <span class="special">Café Soy lent Green</span>,  
the restaurant with something a little different.</p>
```

Once you create a class style, you can apply it to just about any tag on the page. In fact, you can apply the same class to different tags, so you can create a `.special` style with a specific font and color and apply it to `<h2>`, `<p>`, and `<ul>` tags.

One Tag, Multiple Classes

Not only can you apply the same class to different tags, you can also apply multiple classes to the same tag. While it may sound like a lot of extra work to create multiple classes and add multiple class names to the same tag, it's a common approach.

Here's an example of when you might apply multiple classes to the same tag: Imagine that you're designing an interface to manage a visitor's shopping cart. The interface requires a variety of buttons, each of which does something different. One button may be used to delete a product from the shopping cart, another button to add an item, and a third button to edit the quantity.

Being a good designer, you want the buttons to share some similarities, like rounded corners and the same font, but also have their own unique looks: red for the delete button, green for the add button, and so on. To achieve consistency and uniqueness, you can create two classes. One class will apply to all of the buttons, and the other classes will apply to certain types of buttons.

To start, you'd create a `.btn` (short for button) class:

```
.btn {  
  border-radius: 5px;  
  font-family: Arial, Helvetica, serif;  
  font-size: .8 em;  
}
```

Then you could create additional classes for each type of button:

```
.delete {  
    background-color: red;  
}  
.add {  
    background-color: green;  
}  
.edit {  
    background-color: grey;  
}
```

Then, by applying more than one class to a tag, you can combine styles and create both a consistency across the buttons and a unique appearance for each type of button:

```
<button class="btn add">Add</button>  
<button class="btn delete">Delete</button>  
<button class="btn edit">Edit</button>
```

Web browsers and HTML have no problem handling multiple classes applied to a single element. In the HTML tag, just add the `class` attribute, and, for the value, add each class name, separated by a space. The browser will combine the properties from the various classes and apply the final, combined set of styles to the element. So in the current example, all of the buttons will have rounded corners and use the Arial font at .8 ems. But the Add button will be green, the Delete button red, and the Edit button gray.

The benefit of this approach is that if you decide that buttons should no longer have rounded corners, or they should use a different font, you only need to change the `btn` style to update the look of each of the buttons. Likewise, if you decide that the Edit button should be yellow instead of gray, changing the `.edit` style will affect only that one button, not any of the others.

■ ID Selectors: Specific Page Elements

CSS reserves the ID selector for *identifying* a unique part of a page, like a banner, navigation bar, or the main content area. Just as with a class selector, you create an ID by giving it a name in CSS, and then you apply it by adding the ID to your page's HTML code. So what's the difference? As explained in the box on page 48, ID selectors have some specific uses in JavaScript-based or lengthy web pages. Otherwise, compelling reasons to use IDs over classes are few.

NOTE

There's a growing trend in the web design community to abandon ID selectors in CSS. The reasoning requires greater knowledge of CSS than you've learned so far. You won't find IDs much in this book, and you'll learn why ID selectors are often not a good idea on page 48.

UP TO SPEED

Divs and Spans

Chapter 1 introduced you to `<div>` and `<span>`, two generic HTML tags that you can bend to your CSS wishes. When there's no HTML tag that exactly delineates where you want to put a class or ID style you've created, use a `<div>` or `<span>` to fill in the gaps.

The `<div>` tag identifies a logical *division* of the page, like a banner, navigation bar, sidebar, or footer. You can also use it to surround any element that takes up a chunk of the page, including headings, bulleted lists, or paragraphs. (Programmer types call these *block-level* elements because they form a complete “block” of content, with line breaks before and after them.) The `<div>` tag works just like a paragraph tag: Type the opening `<div>`, add some text, a photo, or some other content inside it, and then end it with the closing `</div>`.

The `<div>` tag can contain *several* block-level elements, making it a great way to group tags that are logically related, such as the logo and navigation bar in a page's banner or a series of news stories that compose a sidebar. Once grouped in this way, you can apply specific formatting to just the tags inside the particular div or move the entire `<div>`-tagged chunk of content into a particular area, such as the right side of the browser window (CSS can help you control the visual layout of your pages in this manner, as described in Part Three of this book).

For example, say you added a photo and an accompanying caption to a web page. You could wrap a `<div>` tag (with a class applied to it) around the photo and the caption to group both elements together:

```
<div class="photo">
  
  <p>Mom, dad and me on our yearly trip to
  Antarctica.</p>
</div>
```

Depending on what you put in the declaration block, the `.photo` class can add a decorative border, background color, and so on to both photo and caption. Part Three of this book shows you even more powerful ways to use `<div>` tags—including nested divs.

The latest version of HTML includes many block-level tags that work like divs but are intended for more specific types of content. For example, you can use the `<figure>` tag to display an image and a caption instead of using a `<div>`. However, because IE8 doesn't understand those HTML5 tags (see the box on page 12), many designers still use divs when they wish to group several HTML tags into a single unit.

In addition, the new HTML5 tags are intended to add “meaning” to your HTML. For example, the `<article>` tag is used to indicate a self-contained block of text, like an article in a magazine. Not all markup is meaningful, however, so you'll often still use divs simply to group together elements for stylistic reasons.

A `<span>` tag, on the other hand, lets you apply a class or ID style to just *part* of a tag. You can place `<span>` tags around individual words and phrases (often called *inline* elements) within paragraphs to format them independently. Here, a class called `.companyName` styles the inline elements “CosmoFarmer.com,” “Disney,” and “ESPN”:

```
<p>Welcome to <span class="companyName">
CosmoFarmer.com</span>, the parent
company of such well-known corporations
as <span class="companyName">Disney
</span> and <span class="companyName">
ESPN</span>...well, not really.</p>
```

Although web designers don't use ID selectors as much as they once did, it's good to know what they are and how they work. Should you decide to use an ID selector, creating one is easy. Just as a period indicates the name of a class selector, a pound or hash symbol (#) identifies an ID style. Otherwise, follow the exact same naming

rules used for classes (page 43). This example provides a background color and a width and height for the element:

```
#banner {  
  background: #CC0000;  
  height: 300px;  
  width: 720px;  
}
```

Applying IDs in HTML is similar to applying classes but uses a different attribute named, logically enough, `id`. For example, to apply the style above to a `<div>` tag, you'd write this HTML:

```
<div id="banner">
```

Likewise, to indicate that the last paragraph of a page is that page's one and only copyright notice, you can create an ID style named `#copyright` and add it to that paragraph's tag:

```
<p id="copyright">
```

NOTE

As with class styles, you use the `#` symbol only when naming the style in the style sheet. You leave the `#` off when using the ID's name as part of an HTML tag: `<div id="banner">`.

POWER USERS' CLINIC

Proper IDs

ID attributes in HTML have a few powers that class attributes lack. These benefits actually have nothing to do with CSS, so you may never need an ID. But if you're curious:

- One easy way for JavaScript programmers to locate and manipulate parts of a page is by applying an ID to a page element, and then using JavaScript to reference that ID. For example, programmers often apply an ID to a form element like a text box for collecting a visitor's name. The ID lets JavaScript access that form element and work its magic—like making sure the field isn't empty when the visitor clicks Submit.
- IDs also let you link to a specific part of a page, making long web pages quicker to navigate. If you have an alphabetic glossary of terms, then you can use an ID

selector to create links to the letters of the alphabet. When your visitors click "R," they jump immediately to all the "R" words on the page. You don't actually need to create any CSS for this—it works purely with HTML. First, add an ID attribute to the spot on the page you wish to link to. For example, in a glossary you can add an `<h2>` tag with a letter from the alphabet followed by the glossary listings—perhaps in a definition list or a series of paragraphs. Just add an appropriate ID to each of those `<h2>` tags: `<h2 id="R">R</h2>`. To create the link in HTML, add the `#` symbol and the ID name to the end of the URL, followed by the ID name—`index.html#R`. This link points directly to an element with the ID of `#R` on the page *index.html*. (When used this way, the ID behaves just like a named anchor—`<a name="R">R</a>`—in HTML.)

■ Styling Groups of Tags

Sometimes you need a quick way to apply the same formatting to several different elements. For example, maybe you'd like all the headers on a page to share the same color and font. Creating a separate style for each header—h1, h2, h3, h4, and so on—is way too much work, and if you later want to change the color of all of the headers, then you have six different styles to update. A better approach is to use a *group selector*. Group selectors let you apply a style to multiple selectors at the same time.

Constructing Group Selectors

To work with selectors as a group, simply create a list of selectors separated by commas. So to style all the heading tags with the same color, you can create the following rule:

```
h1, h2, h3, h4, h5, h6 { color: #F1CD33; }
```

This example consists of only type selectors, but you can use any valid selector (or combination of selector types) in a group selector. For example, here's a group selector that applies the same font color to the <h1> tag, the <p> tag, any tag styled with the .copyright class, *and* the tag with the #banner ID:

```
h1, p, .copyright, #banner { color: #F1CD33; }
```

TIP

If you want a bunch of page elements to share *some* but not all of the same formatting properties, then you can create a group selector with the shared formatting options and also create individual rules with unique formatting for each individual element. In other words, two (or more) different styles can format the same tag. The ability to use multiple styles to format a single element is a powerful CSS feature. See Chapter 5 for details.

The Universal Selector (Asterisk)

Think of a group selector as shorthand for applying the same style properties to several different page elements. CSS also gives you a sort of über group selector—the *universal* selector. An asterisk (*) is universal selector shorthand for selecting *every single* tag.

For example, say you want all the tags on your page to appear in bold type. Your group selector might look something like the following:

```
a, p, img, h1, h2, h3, h4, h5 ...yadda yadda... { font-weight: bold; }
```

The asterisk, however, is a much shorter way to tell CSS to select *all* HTML tags on the page:

```
* { font-weight: bold; }
```

You can even use the universal selector as part of a descendant selector, so you can apply a style to all the tags that descend from a particular page element. For example, .banner * selects every tag inside the page element to which you've applied the banner class. (You'll read about descendant selectors next.)

Since the universal selector doesn't specify any particular type of tag, it's hard to predict its effect on an entire website's worth of pages all composed of a variety of different HTML tags. To format many different page elements, web page gurus rely on *inheritance*—a CSS trait discussed in depth in the next chapter.

However, some web designers use the universal selector as a way to remove *all* space around block-level elements. As you'll read on page 187, you can add space around an element using the CSS `margin` property, and add space between the border of an element and the content inside using the `padding` property. Browsers automatically add varying amounts of space for different tags, so one way to start with a clean slate and remove all space around tags is with this style:

```
* {  
  padding: 0;  
  margin: 0;  
}
```

■ Styling Tags Within Tags

Choosing whether to style your page with type selectors or class selectors is a tradeoff. Type selectors are fast and easy, but they make every occurrence of a tag look the same, which is fine—if you want every `<h2>` on your page to look exactly like all the rest. Class and ID selectors give you the flexibility to style individual page elements independently, but creating class or ID styles also requires you to add the appropriate class or ID to the HTML tags you wish to style. Not only does this add more work for you, but it also adds code to your HTML file. What you need is a way to combine the ease of type selectors with the precision of classes and IDs. CSS has just the thing—*descendant selectors*.

You use descendant selectors to format a whole bunch of tags in a similar manner (just like type selectors), but only when they're in a particular part of a web page. It's like saying, "Hey you `<a>` tags in the navigation bar, listen up. I've got some formatting for you. All you other `<a>` tags, just move along; there's nothing to see here."

Descendant selectors let you format a tag based on its relationship to other tags. To understand how it works, you need to delve a little bit more deeply into HTML. On the bright side, the concepts underlying descendant selectors help you understand several other selector types, too, as discussed later in this chapter.

NOTE

Descendant selectors can be confusing at first, but they're among the most important techniques for efficiently and accurately applying CSS. Take the time to master them.

The HTML Family Tree

The HTML that forms any web page is akin to a family tree, where the HTML tags represent various family members. The first HTML tag you use on a page—the `<html>`

tag—is like the grandpappy of all other tags. The `<html>` tag surrounds the `<head>` tag and the `<body>` tag, which makes `<html>` the *ancestor* of both. Similarly, a tag inside of another tag is a *descendant*. The `<title>` tag in the following example is the `<head>` tag's descendant:

```
<html>
  <head>
    <title>A Simple Document</title>
  </head>
  <body>
    <h1>Header</h1>
    <p>A paragraph of <strong>important</strong>text.</p>
  </body>
</html>
```

You can turn the above HTML code into a diagram, like the one in Figure 3-4, showing the relationships between the page's tags. First there's the `<html>` tag; it's divided into two sections represented by the `<head>` and `<body>` tags. Those two tags contain other tags that in turn may contain other tags. By seeing which tags appear inside which other tags, you can diagram any web page.

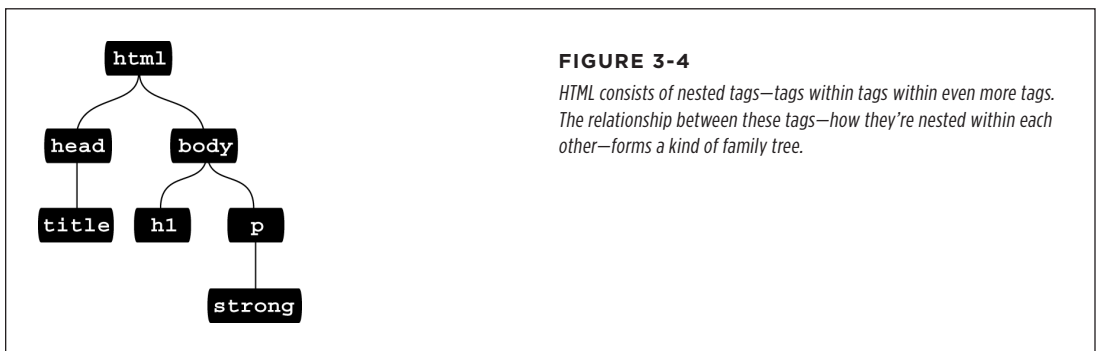


FIGURE 3-4

HTML consists of nested tags—tags within tags within even more tags. The relationship between these tags—how they're nested within each other—forms a kind of family tree.

Tree diagrams help you figure out how CSS sees the relationship of one element on a page to another. Many of the selectors in this chapter, including descendant selectors, rely on these relationships. The most important relationships are:

- **Ancestor.** As explained at the beginning of this chapter, an HTML tag that wraps around another tag is its ancestor. In Figure 3-4, the `<html>` tag is an ancestor of all other tags, while the `<body>` tag is an ancestor for all of the tags *inside* of it—the `<h1>`, `<p>`, and `<strong>` tags.
- **Descendant.** A tag inside one or more tags is a descendant. In Figure 3-4, the `<body>` tag is a descendant of the `<html>` tag, while the `<p>` tag is a descendant of *both* the `<body>` and `<html>` tags.

- **Parent.** A parent tag is the *closest* ancestor of another tag. In Figure 3-4, a parent is the first tag directly connected to and above another tag. Thus, the `<html>` tag is the parent of the `<head>` and `<body>` tags, but of no other tags. And in this diagram, the `<p>` tag is the parent of the `<strong>` tag.
- **Child.** A tag that's directly enclosed by another tag is a child. In Figure 3-4, both the `<h1>` and `<p>` tags are children of the `<body>` tag, but the `<strong>` tag isn't. Since the `<strong>` tag is directly wrapped inside the `<p>` tag, it's a child of the `<p>` tag.
- **Sibling.** Tags that are children of the same tag are called *siblings*, just like brothers and sisters. In an HTML diagram, sibling tags are next to each other and connected to the same parent. In Figure 3-4, the `<head>` and `<body>` tags are siblings, as are the `<h1>` and `<p>` tags.

Thankfully, that's where CSS draws the line with this family metaphor, so you don't have to worry about aunts, uncles, or cousins. (Though rumor has it CSS10 *will* include in-laws.)

Building Descendant Selectors

Descendant selectors let you take advantage of the HTML family tree by formatting tags differently when they appear inside certain other tags or styles. For example, say you have an `<h1>` tag on your web page, and you want to emphasize a word within that heading with the `<strong>` tag. The trouble is, most browsers display both heading tags and the `<strong>` tag in bold, so anyone viewing the page can't see any difference between the emphasized word and the other words in the headline. Creating a type selector to change the `<strong>` tag's color and make it stand out from the headline isn't much of a solution: You end up changing the color of *every* `<strong>` tag on the page, like it or not. A descendant selector lets you do what you really want—change the color of the `<strong>` tag *only when* it appears inside of an `<h1>` tag.

The solution to the `<h1>` and `<strong>` dilemma looks like this:

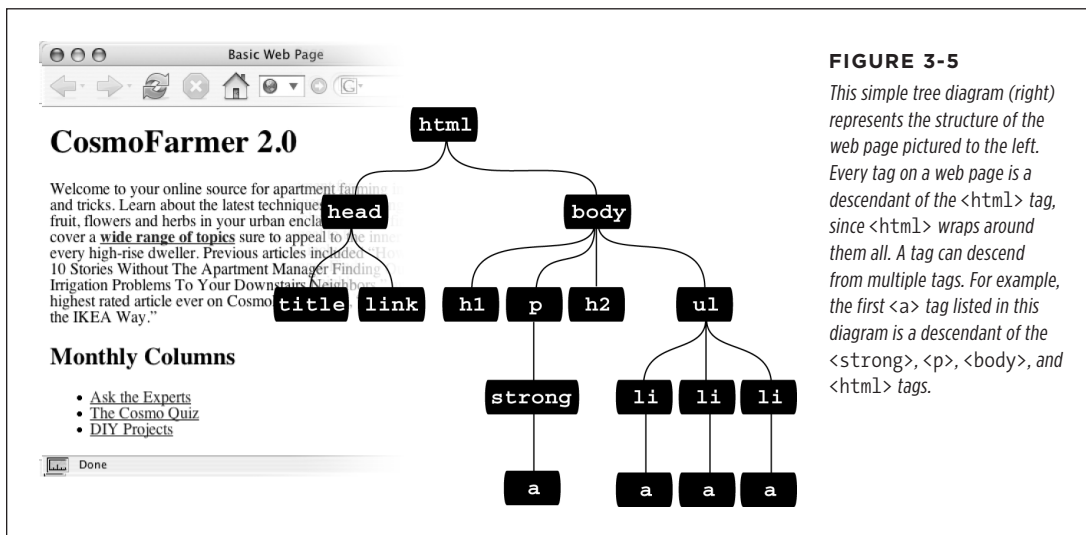
```
h1 strong { color: red; }
```

Here *any* `<strong>` tag inside an `h1` is red, but other instances of the `<strong>` tag on the page aren't affected. You could achieve the same result by creating a class style—.strongHeader, for example—but you'd then have to edit the HTML by adding `class="strongHeader"` to the `<strong>` tag inside the header. The descendant selector approach adds no HTML and no more work beyond creating the style!

Descendant selectors style elements that are nested inside other elements, following the exact same pattern of ancestors and descendants as the tags in the HTML family tree. You create a descendant selector by tacking together selectors according to the part of the family tree you want to format, with the most senior ancestor on the left and the actual tag you're targeting on the far right. For example, in Figure 3-5, notice the three links (the `<a>` tag) inside the bulleted list items and another link

inside the paragraph. To format the bulleted links differently than the other links on the page, you can create the following descendant selector:

```
li a { font-family: Arial; }
```



This rule says, “Format all links (a) that appear inside a list item (li) by using the Arial font.” A descendant selector can contain more than just two elements. The following are all valid selectors for the `<a>` tags inside of the bulleted lists in Figure 3-5:

```
ul li a
body li a
html li a
html body ul li a
```

These four selectors—all of which do the same thing—demonstrate that you don’t have to describe the entire lineage of the tag you want to format. For instance, in the second example, `body li a`, the `ul` isn’t needed. This selector works as long as there’s an `<a>` tag that’s a descendant (somewhere up the line) of an `<li>` tag (which is also a descendant of the `<body>` tag). This selector can just as easily apply to an `<a>` that’s inside an `<em>` tag, that’s inside a `<strong>` tag, that’s inside an `<li>` tag, and so on.

In general, you should use the shortest descendant selector that gets the job done. Because all tags are inside the `<html>` and `<body>` tags, there’s no reason to include them in a descendant selector.

NOTE

The number of selectors you include in a descendant selector affects how the style interacts with other, conflicting styles. This system is called *specificity* and you’ll read all about it in Chapter 5.

You're not limited to just type selectors, either. You can build complex descendant selectors combining different types of selectors. For example, suppose you want your links to appear in yellow only when they're inside an introductory element (which you've designated with a class style named `intro`). The following selector does the trick:

```
.intro a { color: yellow; }
```

Quick translation: Apply this style to every link (`a`) that's a descendant of another tag that has the `intro` class applied to it.

Creating Modules

Descendant selectors are often used to format a *module* of code—that is, a collection of HTML that serves a particular function on a page. For example, say you had a `<div>` on the page, used for listing the latest company news. You might have HTML like this:

```
<div>
  <h2>Our company is great!</h2>
  <p>More information about why our company is so great</p>
  <h2>Another news item</h2>
  <p>Information about the other news item...</p>
  <h2>...and so on...</h2>
  <p>... and so on... </p>
</div>
```

If you simply slap a class on that opening `<div>` tag—`<div class="news">`—then you can create descendant selectors that format the HTML tags within the news section differently. For example,

```
.news h2 { color: red; }
.news p { color: blue; }
```

Now the `<h2>` tags within the news section will be red, and the paragraphs will be blue. It's even possible (and common) to create descendant selectors with multiple class names. For example, say you're building a page that provides a directory of addresses for members of an organization. You might wrap each contact in its own `div`, and further refine the elements inside that `div` with a class like this:

```
<div class="contact">
  <p class="name">John Smith</p>
  <p class="phone">555-555-1234</p>
  <p class="address">1234 Elem St</p>
</div>
```

Then you could create several descendant selectors to style just those contact elements like this:

```
.contact .name { font-weight: bold; }
.contact .phone { color: blue;}
.contact .address { color: red; }
```

NOTE

In a style sheet, you may sometimes see something like this:

```
p.intro
```

This may look like a descendant selector—since there is both an HTML tag and a class—but it's not. There's no space between the `p` and the `.intro`, which means that the `intro` class must be applied specifically to a `<p>` tag (`<p class="intro">`) for this style to work. If you add a space, you get a different effect:

```
p .intro { color: yellow; }
```

This seemingly slight variation selects any tag styled with the `.intro` class, which is itself a descendant of a `<p>` tag. In other words, it doesn't select a paragraph, it selects another tag inside a paragraph. In general, to keep your class styles as flexible as possible, it is better to leave off the HTML tag (in other words use just `.intro` instead of `p.intro`).

Pseudo-Classes and Pseudo-Elements

Sometimes you need to select parts of a web page that don't have tags per se, but are nonetheless easy to identify, like the first line of a paragraph or a link as you move your mouse over it. CSS gives you a handful of selectors for these doohick-eyes—*pseudo-classes* and *pseudo-elements*.

Styles for Links

Four pseudo-classes let you format links in four different states based on how a visitor has interacted with that link. They identify when a link is in one of the following four states:

- **a:link** selects any link that your guest hasn't visited yet, while the mouse isn't hovering over or clicking it. This style is your regular, unused web link.
- **a:visited** is a link that your visitor has clicked before, according to the web browser's history. You can style this type of link differently than a regular link to tell your visitor, "Hey, you've been there already!" (See the box on page 280 for the limitations surrounding this selector.)
- **a:hover** lets you change the look of a link as your visitor passes the mouse over it. The rollover effects you can create aren't just for fun—they can provide useful visual feedback for buttons on a navigation bar.

You can also use the `:hover` pseudo-class on elements other than links. For example, you can use it to highlight text in a `<p>` or `<div>` when your guests mouse over it. In that case, instead of using `a:hover` (which is for links) to add a hover effect, you can create a style named `p:hover` to create a specific effect when someone mouses over any paragraph. If you just want to style tags with a specific class of highlight applied to them, then create a style named `.highlight:hover`.

- **a:active** lets you determine how a link looks *as* your visitor clicks. In other words, it covers the brief nanosecond when someone's pressing the mouse button, before releasing it.

Chapter 9 shows you how to design links by using these selectors to help your visitors click their way around your site.

NOTE

You can live a long, productive life without reading about the selectors in the next few sections. The selectors you've learned so far—tag, class, ID, descendant, group, and so on—let you build absolutely beautiful, functional, and easily maintained websites. If you're ready for the fun stuff—designing web pages—then skip to the tutorial on page 70. You can always finish reading this discussion some cold, rainy night by the fire.

Styling Paragraph Parts

The typographic features that make books and magazines look cool, elegant, and polished didn't exist in the early web era. (After all, when did scientists ever worry about looking cool?) CSS provides two pseudo-elements—`::first-letter` and `::first-line`—that give your web pages the design finesse that print has enjoyed for centuries.

The `::first-letter` pseudo-element lets you create a *drop cap*—an initial letter that jumps out from the rest of the paragraph with bigger or bolder font formatting, as at the beginning of a book chapter.

Styling the `::first-line` of a paragraph in a different color keeps your reader's eye moving and makes text appear appealing and fresh. (If you're intrigued, Chapter 6 is all about formatting text, and page 166 covers these two pseudo-elements in depth.)

NOTE

The latest version of CSS changes the syntax for pseudo-elements. In CSS 2.1, pseudo-elements begin with a single colon, like this:

```
:first-letter
```

CSS3 adds an additional colon to differentiate pseudo-classes like `:hover` from pseudo-elements. So `:first-letter` and `:first-line` are now `::first-letter` and `::first-line`.

More Pseudo-Classes and Pseudo-Elements

The CSS guidelines define several powerful pseudo-class and pseudo-element selectors besides the ones covered so far. Support for these selectors in all but the oldest browsers is very good.

■ :FOCUS

The `:focus` pseudo-class works much like the `:hover` pseudo-class. While `:hover` applies when a visitor mouses over a link, `:focus` applies when the visitor does

something to indicate her attention to a web page element—usually by clicking or tabbing into it. In programming lingo, when a visitor clicks in a text box on a web form, she puts the *focus* on that text box. That click is a web designer’s only clue as to where the visitor is focusing her attention.

The `:focus` selector is mostly useful for giving your visitor feedback, like changing the background color of a text box to indicate where she’s about to type. (Single-line text fields, password fields, and multi-line `<textarea>` boxes are common targets for the `:focus` selector.) This style, for example, adds a light yellow color to any text box a visitor clicks or tabs into:

```
input:focus { background-color: #FFFFCC; }
```

The `:focus` selector applies only while the element is in focus. When a visitor tabs into another text field or clicks anywhere else on the page, she takes the focus—and the CSS properties—away from the text box.

TIP

One good resource for seeing which browsers support which CSS selector is <http://caniuse.com/>.

■ ::BEFORE

The `::before` pseudo-element does something no other selector can: It lets you add content preceding a given element. For example, say you wanted to put “HOT TIP!” before certain paragraphs to make them stand out, like the boxes in this book that say “UP TO SPEED” and “POWER USERS’ CLINIC.” Instead of typing that text in your page’s HTML, you can let the `::before` selector do it for you. This approach not only saves on code, but also if you decide to change the message from “HOT TIP!” to, say, “Things to know,” then you can change every page on your site with one quick change to your style sheet.

First, create a class (`.tip`, say) and apply it to the paragraphs that you want to precede with the message, like so: `<p class="tip">`. Then, add your message text to the style sheet:

```
.tip::before {content: "HOT TIP!" }
```

Whenever a browser encounters the `tip` class it dutifully inserts the text “HOT TIP!” just before the tag with that class.

The technical term for text you add with this selector is *generated content*, since web browsers create (generate) it on the fly. In the page’s HTML source code, this material doesn’t exist. Whether you realize it or not, browsers generate their own content all the time, like the bullets in bulleted lists and numbers in ordered lists. If you want, you can even use the `:before` selector to define how a browser displays its bullets and numbers for lists.

■ ::AFTER

Exactly like the `::before` selector, the `::after` pseudo-element adds generated content—but after the element, not before. You can use this selector, for example, to add closing quotation marks (") after quoted material.

NOTE

Both `::before` and `::after` are pseudo-elements like `::first-line` and `::first-letter`. The latest version of CSS adds double colons to pseudo-elements.

■ ::SELECTION

This CSS3 selector refers to items that a visitor has selected on a page. For example, when a visitor clicks and drags over text, the browser highlights that text, and the visitor can then copy the text. Normally, browsers add a blue background behind the text. However, you can control the background color and text color by defining this selector. For example, if you wanted to make selected text white with a violet background you could add this style to a page's style sheet:

```
::selection {  
    color: #FFFFFF;  
    background-color: #993366;  
}
```

The only properties you can set with this selector are `color` and `background-color`, so you can't go wild and change the font size, fonts, margins, or other visual changes that would surely drive your site visitors crazy. (Thanks for protecting us from ourselves, CSS!)

NOTE

There's no single-colon version of the selection pseudo-element, so you must use the double colon.

The `::selection` CSS pseudo-element applies styles to the part of a document that has been highlighted by the user (such as clicking and dragging the mouse across text).

If you really want to go crazy, you can specify a different background color just for text selected inside a particular element. For example, to make only text inside

paragraphs red with a pink background, just add the `p` element selector before `::selection`, like this:

```
p::selection {  
    color: red;  
    background-color: pink;  
}
```

TIP Learning how to write selectors can sometimes feel like learning hieroglyphics. To translate a selector into straightforward language, visit Mozilla Firefox at https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_selectors. This great resource lets you type in a selector, and then it spits out a clear description of which page elements on a page the style affects.

■ Attribute Selectors

CSS provides a way to format a tag based on any HTML attributes it has. For example, say you want to place borders around the images on your page—but only around the important photos. You don't want to include your logo, buttons, and other little doodads that also have an `<img>` tag. Fortunately, you realize that you've given all the photos descriptions using the `title` attribute, which means you can use an *attribute selector* to identify just the important images.

With attribute selectors, you can single out tags that have a particular property. For example, here's how to select all `<img>` tags with a `title` attribute:

```
img[title]
```

The first part of the selector is the name of the tag (`img`) while the attribute's name goes in brackets: `[title]`.

CSS doesn't limit attribute selectors to tag names: You can combine them with classes, too. For example, `.photo[title]` selects every element with the `.photo` class style and an HTML `title` attribute.

To get more specific, you can select elements that not only share a particular attribute, but also have an exact value set for that attribute. For example, when you want to highlight links that point to a particular URL, create an eye-catching attribute selector, like so:

```
a[href="http://www.cafesoylentgreen.com"]{  
    color: green;  
    font-weight: bold;  
}
```

Adding a value to an attribute selector is very useful when working with forms. Many form elements have the same tag, even if they look and act differently. The checkbox, text box, submit button, and other form fields all share the `<input>` tag. The type

attribute's value is what gives the field its form and function. For example, `<input type="text">` creates a text box, and `<input type="checkbox">` creates a checkbox.

To select just text boxes in a form, for example, use this selector:

```
input[type="text"]
```

The attribute selector is very versatile. It lets you not only find tags that have a specific value for an attribute (for example, find all form fields with a type of checkbox) but even select elements with an attribute value that *begins with*, *ends with*, or *contains* a specific value. While this might sound like overkill, it's actually quite handy.

The `[attribute^="value"]` selector is used to select elements with the specified attribute, whose value starts with the specified value.

The following example selects all elements with a class attribute value that starts with "top":

Note: The value does not have to be a whole word!

```
[class^="top"] {  
  background: yellow;  
}
```

Similarly, there are times when you want to select an element with an attribute that *ends* in a specific value.

The `[attribute$="value"]` selector is used to select elements whose attribute value ends with a specified value.

The following example selects all elements with a class attribute value that ends with "test":

Note: The value does not have to be a whole word!

```
[class$="test"] {  
  background: yellow;  
}
```

Finally, you can even select elements with attributes that contain another value. The `[attribute*="value"]` selector is used to select elements whose attribute value contains a specified value.

The following example selects all elements with a class attribute value that contains "te":

Note: The value does not have to be a whole word!

```
[class*="te"] {
  background: yellow;
}
```

CSS `:is()` Pseudo-class

https://www.w3schools.com/cssref/selector_is.php

CSS `:has()` Pseudo-class

https://www.w3schools.com/cssref/selector_has.php

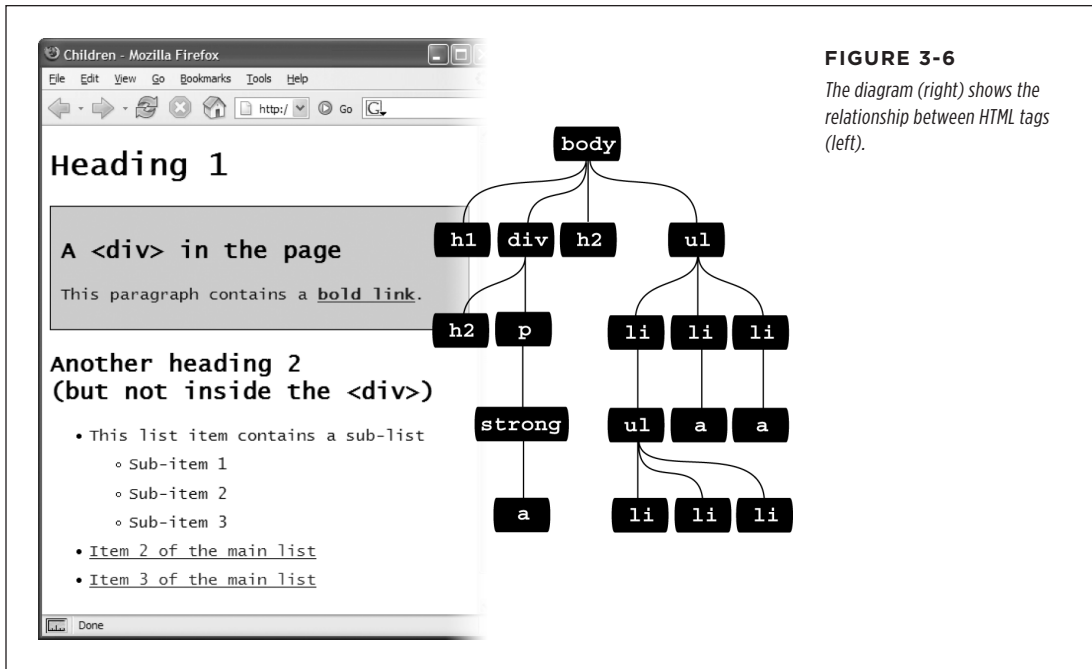
CSS `:where()` Pseudo-class

https://www.w3schools.com/cssref/selector_where.php

■ Child Selectors

Similar to the descendant selectors described earlier in this chapter, CSS lets you format the children of another tag with a *child* selector. The child selector uses an additional symbol—an angle bracket (`>`) to indicate the relationship between the two elements. For example, the selector `body > h1` selects any `<h1>` tag that's a child of the `<body>` tag.

Unlike a descendant selector, which applies to *all* descendants of a tag (children, grandchildren, and so on), the child selector lets you specify which child of which parent you mean. For example, in Figure 3-6, there are two `<h2>` tags. Using a plain old descendant selector—`body h2`—selects both `<h2>` tags. Even though both `<h2>` tags are inside of the `<body>` tag, only the second one is a child of the `<body>` tag. The first `<h2>` is directly inside of a `<div>` tag, so its parent is the `<div>`. Since the two `<h2>` tags have different parents, you can use a child selector to get at them individually. To select only the first `<h2>` tag, your child selector looks like this: `body > h2`. If you want the second `<h2>` tag, then you must use this child selector instead: `div > h2`.

**FIGURE 3-6**

The diagram (right) shows the relationship between HTML tags (left).

CSS also includes some very specific pseudo-classes for selecting child elements. They let you fine-tune your selectors for many different arrangements of HTML.

■ :FIRST-CHILD

Going back to the HTML family tree analogy for a moment, recall what a child tag is: any tag directly enclosed by another tag. (For example, in Figure 3-6, `<h1>`, `<div>`, `<h2>`, and `<ul>` are all children of the `<body>` tag.) The `:first-child` pseudo-element lets you select and format just the *first* of however many children an element may have.

In Figure 3-6, if you want to select the first `<h1>` tag on the page, create a selector like this:

```
h1:first-child
```

This selector applies to any `<h1>` tag that is a first child. In Figure 3-6, the results are obvious: There's only one `<h1>` tag, and it's the first tag on the page. Therefore, it's a child of the `<body>` element. However, `:first-child` can be confusing. For example, if you change the `<h2>` tag inside the `<div>` pictured in Figure 3-6 to an `<h1>` tag, then `h1:first-child` would select both the `<h1>` directly inside the `<body>` tag and the `<h1>` inside the `div` (since that `<h1>` is the first-child of the `div`).

■ :LAST-CHILD

This is like `:first-child` discussed earlier, but just with the last child of an element. For example, to style the last item in a list, you use the `li:last-child` selector (see Figure 3-7).

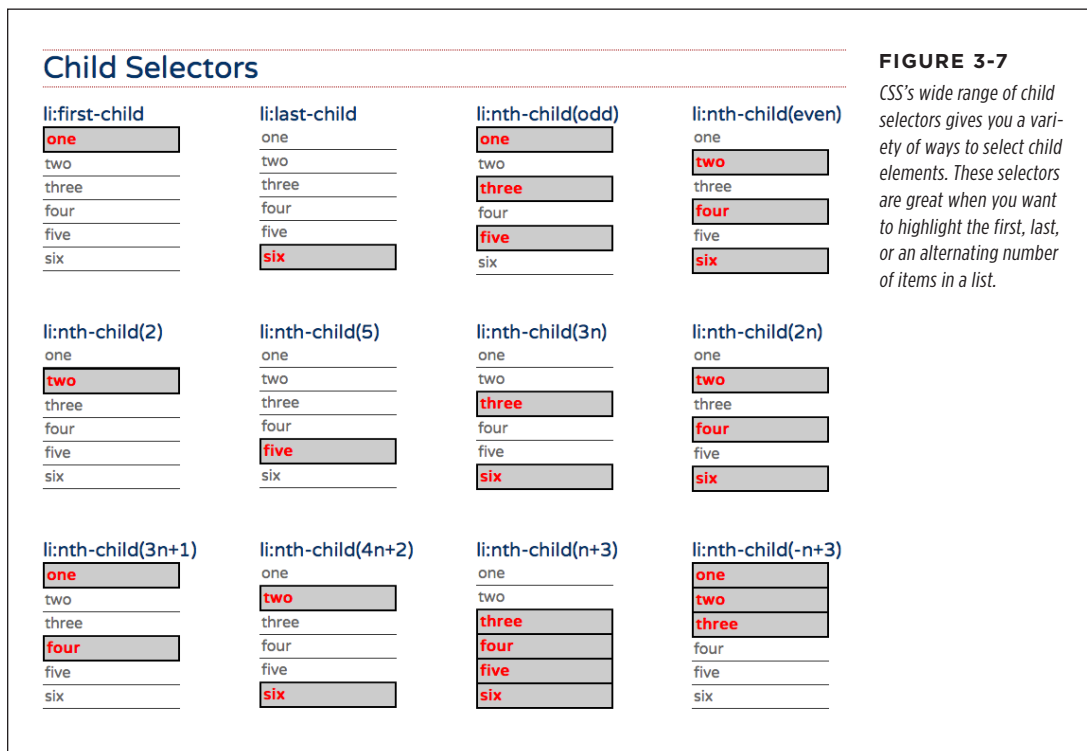


FIGURE 3-7

CSS's wide range of child selectors gives you a variety of ways to select child elements. These selectors are great when you want to highlight the first, last, or an alternating number of items in a list.

■ :ONLY-CHILD

There's also a selector for an element that's the only child of another element. For example, say you have this style in your style sheet:

```
p:only-child {
  color: red;
}
```

This style sets the text color to red, but only if there's a single paragraph inside the element. For example, if you have a `<div>` tag with three paragraphs inside it, then this style wouldn't apply—because there are three children inside that div. However, if you deleted two of the paragraphs, leaving just one paragraph in the div, that paragraph would then be red.

To make things more confusing, however, this style only works when the specific tag is the only child inside another tag. In other words, it's not enough that the tag is the only one of its kind. If there's another tag that's a sibling to the specified tag,

this selector won't work. If you added a `<ul>` tag inside `<div>` along with the `<p>`, then the paragraph is no longer an only child. That `<ul>` tag is also a child, so the `p:only-child` would no longer apply!

■ :NTH-CHILD

This complex selector is very useful. With it, you can easily style every other row in a table, every third list item, or any combination of alternating child elements (see Figure 3-7). This selector requires a value to determine which children to select. The easiest option is a keyword—either `odd` or `even`—which lets you select alternating odd or even child elements. For example, if you want to provide one background color for each even row in a table and another color in the background of each odd-numbered row, you can write two styles like this:

```
tr:nth-child(odd) { background-color: #D9F0FF; }
tr:nth-child(even) { background-color: #FFFFFF; }
```

Now that's a really simple way to color alternating table rows (see Figure 3-8). But `:nth-child()` has even more power up its sleeve.

You can pinpoint a specific child by supplying a number. For example, if you want to style the fifth item in a list, you'd provide the number 5 to the `:nth-child` selector, like this:

```
li:nth-child(5)
```

That style selects only a single child. If you want to select, say, every third item in a list, use the number (3, in this case) followed by the letter `n`, like this:

```
li:nth-child(3n)
```

Then `n` indicates a multiplier, so `3n` is every third child beginning with the third child (see Figure 3-7).

But what if you want to select every third child element in a series, starting with the second child element? For example, suppose you want to highlight every third table cell (`<td>` tag) inside a row, starting with the second table cell (see Figure 3-8). Here's a style to achieve that:

```
td:nth-child(3n+2) { background-color:#900; }
```

The number before the `n` represents a multiple: `3n` means every third element, `4n` means every fourth element, and so on. The plus sign followed by a number (`+2` in this example) indicates which element to start at, so `+2` means start at the second child element, while `+5` means start at the fifth child element. So `:nth-child(5n+4)` selects every fifth child element starting at the fourth child element.

You can even use a negative `n` value, which will cycle through the child elements *backwards*. For example the last list in Figure 3-7 uses the following selector:

```
li:nth-child(-n+3)
```


which translates to “start at the third list item, then select every list item before it.” As you can see, the `nth-child` selector is confusing, but powerful enough to let you select an endless variety of child elements.

Alternating Table Rows

FIGURE 3-8

Table-stripping the easy way: with child selectors. You can even stripe alternating columns by targeting every other `<td>` tag within a row, or, as in this case, every third column beginning with the second one. Now that's precision!

Child Type Selectors

CSS includes a selector that works much like the child selectors in the previous section but applies to children with a specific type of HTML *tag*. For example, say you want to format the first paragraph inside a sidebar in a particular way, but on some pages, that sidebar starts with an `<h2>` tag, and on other pages, it starts with a `<p>` tag. You can't use `:first-child` to select that paragraph, since in some cases it's the *second* child (following the `<h2>`). However, it's always the first paragraph (`<p>` tag) in that sidebar, even if other tags come before it, so you can select it with a type selector called **`:first-of-type`**.

NOTE `:last-child`, `:first-of-type`, and `:nth-child()` are supported in all modern browsers

■ **`:FIRST-OF-TYPE`**

This selector works just like `:first-child`, but applies to a child that has a particular tag. For example, say you have a sidebar element with the class `sidebar`. To style the first paragraph in that sidebar, use this selector:

```
.sidebar p:first-of-type
```

Notice the `p` in `p:first-of-type`. It indicates the tag you're going to format.

■ **`:LAST-OF-TYPE`**

This selector works like `:last-child`, but applies to the last instance of a particular type of tag. For example, if you want to format the last paragraph in the sidebar `div` in a particular way, but you're not sure whether there are other tags coming after the paragraph (like a bulleted list, headline, or image). Here's the style:

```
.sidebar p:last-of-type
```

NOTE

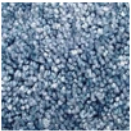
Remember, these type selectors also have to be children of a particular tag. So `p:first-of-type` means “the first child that is a paragraph tag.”

■ :NTH-OF-TYPE

Works like `:nth-child()`, but it applies to alternating children that have a specific tag. You may find this selector handy if you have something like a big paragraph of text that’s peppered with photos. The `<img>` tag is an inline tag, so you can have a `<p>` tag with a bunch of `<img>` tags inside it. And say you want to alternately float the images left and right, as shown in Figure 3-9. You can do so with these two styles:

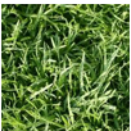
```
img:nth-of-type(odd) { float: left; }
img:nth-of-type(even) { float: right; }
```

Type Selectors



Ullamco laboris nisi sed do eiusmod tempor incididunt excepteur sint occaecat. In reprehenderit in voluptate velit esse cillum dolore ut labore et dolore magna Ullamco laboris nisi mollit anim id est laborum. Cupidatat non proident, excepteur sint occaecat qui officia deserunt. Velit esse cillum dolore ut aliquip ex ea commodo consequat. Eu fugiat nulla pariatur. In reprehenderit in voluptate. In reprehenderit in voluptate lorem ipsum dolor sit amet, caliqua. Sunt in

culpa quis nostrud exercitation mollit anim id est laborum. Sed do eiusmod tempor incididunt lorem ipsum dolor sit amet, excepteur sint occaecat. Ut aliquip ex ea commodo consequat. Ut enim ad minim veniam, consectetur adipisicing elit, sed do eiusmod tempor



incididunt. Cupidatat non proident. Velit esse cillum dolore sed do eiusmod tempor incididunt ut enim ad minim veniam. Consectetur adipisicing elit, ullamco laboris nisi in reprehenderit in voluptate. Duis aute irure dolor quis nostrud exercitation eu fugiat nulla pariatur. Sed do eiusmod tempor incididunt consectetur adipisicing elit, ut labore et dolore magna aliqua. In reprehenderit in voluptate

ut aliquip ex ea commodo consequat. Sunt in culpa sed do eiusmod tempor incididunt duis aute irure dolor. Ut enim ad minim veniam, lorem ipsum dolor sit amet, ullamco laboris nisi. Sed do eiusmod tempor incididunt eu fugiat nulla pariatur. Quis nostrud exercitation consectetur adipisicing elit, in reprehenderit in voluptate. upidatat non proident. Ut enim ad minim veniam, consectetur adipisicing elit, sunt in culpa.

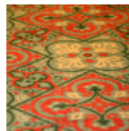


FIGURE 3-9

With the `:nth-of-type()` selector, you can easily select every other image inside a tag, alternating between left and right alignment.

As you can see, you use the same keywords (odd or even) and formula (here, $2n+1$) for `:nth-of-type()` as you do for `:nth-child()`.

In fact, you can use `:nth-of-type()` to select alternating table rows as well:

```
tr:nth-of-type(odd) { background-color: #D9F0FF; }
tr:nth-of-type(even) { background-color: #FFFFFF; }
```

In the case of CSS selectors, there's always more than one way to skin an HTML tag—there are usually five or more!

FREQUENTLY ASKED QUESTION

Making Lists Look Great

When would I ever use a child selector? Just from reading this chapter, I already know enough selectors to get at just about any page element, so why learn another?

There's one design challenge where child selectors can't be beat—and it comes up in more websites than you think. Any time you have an unordered list with one or more unordered lists nested inside (as in Figure 3-6), you can use child selectors to visually organize these categories and subcategories of information. You can format the first level of list items one way, and the second level of list items another way. Content presented in this manner looks neat, professional, and readable (and your visitors will love you for it).

First, create a class style for the outermost nested level in your list and call it, say, `.mainList`. For this top level, you might use a sans-serif font, a little larger than your other text, perhaps in bold or a different color. Subsequent categories can be smaller, in a serif font like Times for easiest reading. When

you have a lot of text, styling each subcategory level a bit differently helps visually orient your visitors in the material.

Apply the `.mainList` class style to the first `<ul>` tag: `<ul class="mainList">`. Then use a child selector (`ul.mainList > li`) to select just the first set of list items, and add your desired text styling for the first subcategory. This styling applies to the `<li>` tags that are children of the `<ul>` tag with the `.mainList` style applied to it. Then, to style the child `<li>` tags of any subsequent nested `<ul>` tags, use this selector, `ul.mainList > li > ul > li`. (A descendant selector like `ul li`, by contrast, selects the list items of all unordered lists on the page—nested ones and all.)

You'll need to keep a concept you'll learn in the next chapter—inheritance—in mind. Basically, certain CSS properties applied to one tag are inherited by tags inside them. So even if you use a child selector to target the children of one tag, properties may pass on to other tags inside that child. The `:not()` selector, discussed on page 69, is one way to avoid this.

Siblings

Parent-child relationships aren't the only ones in the HTML family tree. Sometimes you need to select a tag based not on its parent tag but on its surrounding siblings—the tags that share a common parent. A tag that appears immediately after another tag in HTML is called an *adjacent sibling*. In Figure 3-6, the `<div>` tag is the adjacent sibling of the `<h1>` tag, the `<p>` tag is the adjacent sibling of the `<h2>` tag, and so on.

Using an adjacent sibling selector, you can, for example, give the first paragraph after each heading different formatting from the paragraphs that follow. Suppose you want to remove the margin that appears above that `<p>` tag so that it sits right below the heading without any gap. Or perhaps you want to give the paragraph a distinct color and font size, like a little introductory statement.

The adjacent sibling selector uses a plus sign (+) to join one element to the next. So to select every paragraph following each `<h2>` tag, use this selector: `h2 + p` (spaces are optional, so `h2+p` works as well). The last element in the selector (`p`, in this case) is what gets the formatting, but only when it's directly after its brother `<h2>`.

There's another sibling selector called the *general sibling combinatory selector* (say that three times fast). It's simply a ~ (tilde) and it means "select all siblings of this type." For example, while `h2 + p` selects a single `<p>` tag that immediately follows an `<h2>` tag, `h2 ~ p` selects *all* `<p>` tags that are siblings (that is, on the same level) of the `h2`. To be honest, you may never find a good use for this selector, but CSS is nothing if not thorough.

■ The :target Selector

The `:target()` selector is fun. It can let you create some really interesting effects, like applying a style to a page element *after* another element is clicked. That's the kind of interactivity usually reserved for JavaScript. The `:target()` selector relies on a particular use of ID attributes, as described in the box on page 48; you use IDs to link to a particular spot on a page.

For example, say you have a web page called *index.html*. On that page, you have a div with the ID of `signupForm`. If you have a link on this page (or another page) that points to `index.html#signupForm`, then the web browser jumps to that div. If that div is at the bottom of a very long web page, then the browser scrolls the page until that div is in view. These are sometimes called *in page* links, and they're often used on glossary-style web pages where visitors can click a letter and jump to the spot on the page where the glossary words beginning with that letter appear.

But you don't have to use this feature to jump to another spot on the page. Whenever a URL in the browser's location bar includes a # followed by an ID, the element with that ID becomes the target, so you can apply a specific style to an element only when the ID for that element is in the URL.

Here's an example of HTML showing how the `:target` selector works. Imagine this code appears near the top of a web page:

```
<button>
  <a href="#signupForm">Sign up for our newsletter</a>
</button>
<form id="signupForm">
  <label for="email">What's your email address?</label>
  <input type="email" id="email">
  <input class="btn" type="submit" value="Sign up">
</form>
```

When a visitor clicks the link—the `<a>` tag—the form is targeted. In other words, you can apply one style to the form in its regular state and another style when a visitor clicks that link. For example, you could start off by hiding the form (you'll learn more about CSS visibility properties on page 439) using this rule:

```
#signupForm {
  display: none;
}
```

This rule hides the form, so when the page loads, the visitor can't see it. But when the visitor clicks the "Sign up for our newsletter" button and the form is targeted, you can show it like this:

```
#signupForm:target {
  display: block;
}
```

In other words, if the URL in the browser's location bar is just the filename—`index.html`, for example—then the browser applies the first style and hides the form. But if the location looks like this—`index.html#signupForm`—then the target style applies and the form is made visible. Cool stuff.

NOTE For a really cool use of the `:target` selector, check out this all-CSS gallery: <http://benschwarz.github.io/gallery-css/>.

■ The :not() Selector

The `:not()` selector, also called the *negation pseudo-class*, lets you select something that's not something else. For example, say you apply a class to a paragraph—`<p class="classy">`—and create a CSS class selector to format it, like this:

```
.classy { color: red; }
```

But what if you want to select all paragraphs *except* the `classy` paragraphs? That's where the `:not()` selector comes in. You put a CSS selector inside the parentheses to indicate what you *don't* want to select. For example:

```
p:not(.classy) { color: blue; }
```

This style makes the text color blue for all paragraphs that don't have the `classy` class.

The `:not()` selector can come in handy when you're using the attribute selectors discussed on page 59. For example, you saw on page 59 that you can use an attribute selector to pick all links that point outside your website, like this:

```
a[href^="http://"]
```

As you've probably noticed, that selector doesn't specifically select all links that point outside your site; it simply selects all links that use absolute URLs—that is, URLs beginning with `http://`. For many sites, that's the same thing. Many sites use document- or root-relative links to point to other pages within the site, and absolute URLs to link to other sites. However, in some cases, you may use absolute URLs to point to pages within your site.

For example, many content management systems (WordPress, for example) use absolute URLs to point to blog posts within the site. In this case, if you want to style links that point outside your site, you need to refine the basic attribute selector by also employing the `:not()` selector. For example, say your site's domain name is

mysite.com. To select links that point outside your site, you'll want to select all absolute links that *don't* point to the domain *mysite.com*. Here's how you would do that:

```
a[href^="http://"]:not([href^="http://mysite.com"])
```

Translated into English, this selector says “Select all links whose href attribute begins with *http://*, but *not* ones that begin with *http://mysite.com*.” As you'll recall from page 60, in an attribute selector, ^= means “begins with.” A shorter way to write the same thing would be this:

```
a[href^="http://"]:not([href*="mysite.com"])
```

In an attribute selector, *= means “contains,” so this line would exclude any absolute URL that contains *mysite.com*. This would include *http://www.mysite.com* as well as *http://mysite.com*.

There are some limitations to the `:not()` selector:

- You can only use *simple selectors* with the `:not()` selector. In other words you can use element selectors (like `html` or `p`), the universal selector (`*` [see page 49]), classes (`.footer`, for example), IDs (`#banner`, for example), or pseudo-classes (`:hover`, `:checked`, `:first-child`, and so on). So the following are all valid:

```
.footnote:not(div)
img:not(.portrait)
div:not(#banner)
li:not(:first-child)
```

- You can't use descendant selectors (like `div p a`, pseudo-elements (like `::first-line`), group selectors, or combinators (like the adjacent sibling selector `h2 + p`).
- You can't string multiple `:not()` selectors together. For example, the following is invalid:

```
a[href^="http://"]:not([href*="google.com"]):not([href="yahoo.com"])
```

In other words, you can only use `:not()` once with a selector.

Tutorial: Selector Sampler

In the rest of this chapter, you'll create a variety of selector types and see how each affects a web page. This tutorial starts with the basic selector types and then moves on to more advanced styles.

To get started, you need to download the tutorial files located on this book's companion website at https://github.com/sawmac/css_mm_4e. Click the tutorial link and download the files. All of the files are enclosed in a zip archive, so you'll need to unzip them first. The files for this tutorial are contained inside the folder named *03*.

INDIGNANTLY ASKED QUESTION

Keeping It Internal

Hey, what's up with the internal style sheet in this tutorial? Chapter 2 recommends using external style sheets for a bunch of reasons.

Think you're pretty smart, eh? Yes, external style sheets usually make for faster, more efficient websites, for all the reasons mentioned in Chapter 2. However, internal style sheets make your life easier when you're designing a single page at a time, as in this tutorial. You get to work in just one web page file instead of flipping back and forth between the external style sheet file and the web page.

Furthermore, you can preview your results without constantly refreshing your browser's cache; flip back to the box on page 25 for more on that quirkiness.

So, yes, you should use external style sheets for your sites. And if you were going to use the styles you created in this tutorial on more than just the single tutorial HTML file, you would. But just to keep things fast and simple as you learn CSS, you'll use a single HTML file and an internal style sheet for this exercise. But, that was an excellent question! Keep up the good work.

1. In your favorite text editor, open **03→selector_basics.html**.

This page is made of very basic HTML tags (see Figure 3-10). But you'll liven things up in this tutorial. First, you'll link to a Google font—the same one you used back on page 36.

2. In the empty line below the closing `</title>` tag, type:

```
<link href='http://fonts.googleapis.com/css?family=Varela+Round'
rel='stylesheet'>
```

As described on page 36, this tag links to an external style sheet that Google hosts on its web servers. It downloads the Varela Round font and so you can use it on the page. (You'll learn more about using web fonts like this one from Google on page 140.) Next, you'll add the internal style sheet.

3. After the `<link>` tag you added in the last step, hit Return and type `<style>`. Press Enter twice and then type `</style>`.

These are the opening and closing style tags—it's a good idea to type both tags at the same time, so you don't accidentally forget to add the closing `</style>` tag. Together, these two tags tell a web browser that the information between them is Cascading Style Sheet instructions. The HTML should now look like this (the stuff you added is in bold):

```
<title>Selector Basics</title>
<link href='http://fonts.googleapis.com/css?family=Varela+Round'
rel='stylesheet'>
<style>

</style>
```

Type selectors—like the one you’re about to create—are the most basic kind of selector. If you completed the tutorial in the last chapter, you’ve already created a few. Here, you’ll add a background color to the page.



CSS: The Missing Manual

The Amazing World of CSS

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt.

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt. Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit, sed quia non numquam eius modi tempora incidunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur?

NOTE: Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur?

Who Knew CSS Had Such Power?

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt. Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit, sed quia non numquam eius modi tempora incidunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur?

NOTE: Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur?

Not Me!

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt. Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit, sed quia non numquam eius modi tempora incidunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur?

Me Neither!

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt. Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit, sed quia non numquam eius modi tempora incidunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam corporis suscipit laboriosam, nisi ut aliquid ex ea commodi consequatur? Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui dolorem eum fugiat quo voluptas nulla pariatur?

FIGURE 3-10

Plain HTML looks cold and monotonous in a web browser. But with a little CSS, you can turn drab (shown here) into fab (Figure 3-11) in 31 easy steps.

4. Click between the opening and closing style tags you just added and type body { , hit Enter twice, and type the closing }.

It’s a good idea to always add the closing brace immediately after typing the opening brace, just so you don’t forget. To create a tag selector, simply use the name of the HTML tag you wish to format. This style applies to the <body> tag. Now you can set the background color and the margin of space around the page.

5. Click between `<body>` style's opening and closing braces (`{ }`) and add the following three CSS properties to supply the style's formatting—color, size, font, and left indent (as set by the padding):

```
body {  
  background-color: rgb(50,122,167);  
  padding: 0 20px 20px 20px;  
  margin: 0;  
}
```

Press Enter to place each CSS property on its own line. It's also a good idea to visually organize your CSS code by indenting each property with the Tab key (some designers use two spaces instead of a tab—it's up to you).

The properties here change the background color of the page: `rgb()` is one way to specify a color's red, green, and blue values. In this case, the color is a dark blue. This background makes the black text hard to read, so you need to change the color of the paragraph tags.

NOTE

These property names and their values may look unfamiliar. For now, just type them as is so you can get a taste of padding and margins work. You'll learn much more about these properties in Chapter 6.

6. Add another style below the body style you just created:

```
p {  
  color: rgba(255,255,255,.6);  
  font-size: 1em;  
  font-family: "Varela Round", Arial, Helvetica, sans-serif;  
}
```

With four CSS properties, this style supplies formatting for all paragraphs (all `<p>` tags)—color, size, and font. This time, the color is determined by an `rgba()` color value. With an extra “a” after the usual “rgb,” this versatile value lets you create a color that's partially transparent. In this case, you've set the paragraph's text to white (which is 255,255,255 in rgb), but only 60 percent opaque (that's the .6 part). A bit of the blue background color shows through, so the paragraph text appears light blue.

Time for a look-see.

7. Open the page in a web browser to preview your work.

Unless you tinker with the preference settings, most browsers display black text in a standard serif font like Times. If your CSS style works properly, then you should see seven paragraphs using the Varela Round font in a light blue color.

Creating a Group Selector

Sometimes you'll want several different elements on a page to share the same look. For instance, you may want all your headings to have the same font and color for a

consistent style. Instead of creating separate styles and duplicating the same property settings for each tag—`<h1>`, `<h2>`, and so on—you can group the tags together into a single selector.

1. Return to your text editor and the [selector_basics.html](#) file.

Let's add a new style below the `<p>` tag style you just created.

2. Click at the end of the closing brace of the `p` tag selector, press Enter to start a new line, and the following code:

```
h1, h2, h3 {  
  
}
```

As explained earlier in this chapter, a group selector is simply a list of selectors separated by commas. This rule applies the same formatting, which you'll add next, to all `<h1>`, `<h2>`, and `<h3>` tags on the page.

3. Click in the empty line between the opening `{` and closing `}` and add five CSS properties:

```
color: rgb(255,255,255);  
font-family: Arial, "Palatino Linotype", Times, serif;  
border-bottom: 2px solid rgb(87,185,178);  
padding-top: 10px;  
padding-bottom: 5px;
```

There's a lot going on here, but basically you're setting the color and font type for the headlines, adding a border line below the headlines for visual interest, and controlling the top and bottom spacing by using the `padding` property. The `padding` property adds space from the edges of an element without affecting a background or border—you're adding a bit of space above the headline, and inserting a bit of space between the bottom of that text and the border line below it.

4. Save the file, and preview it in a web browser.

The `<h1>` heading near the top of the page and the `<h2>` and `<h3>` headings lower on the page all have the same font and font color as well as a greenish-blue border below them (see Figure 3-11). The `<h1>` tag looks a bit small, but you can easily bump up its size.

5. Go back to your text editor and the [selector_basics.html](#) file. Add another style below the group selector style you just created:

```
h1 {  
    font-size: 2em;  
}
```

This style increases the size of the font. An `em` is the default browser font-size, so `2em` is twice the normal text size. Notice, as well, that it's possible to have

more than one style apply to an element at the same time—the `h1`, `h2`, `h3` rule and the `h1` rule in this case. Both the group selector and this new type selector apply to `h1` tags on this page. This process is the *cascade* in Cascading Style Sheets. You'll learn all about how styles interact in Chapter 5.

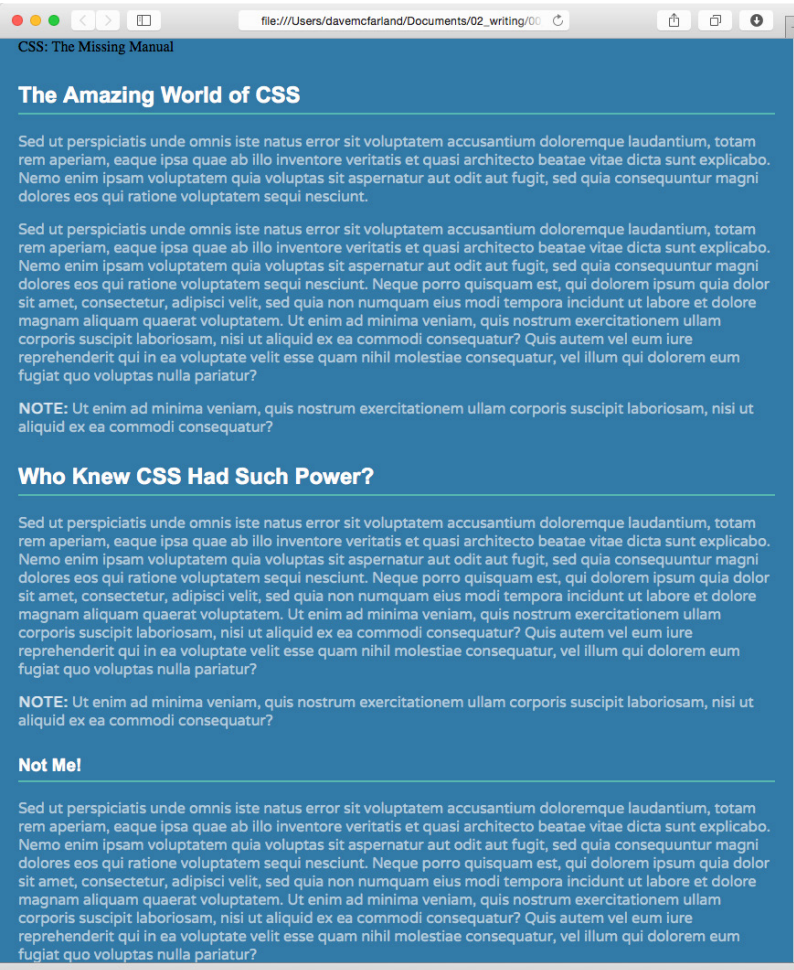


FIGURE 3-11

A simple type selector can completely transform the appearance of every instance of a tag, making quick work of styling all the paragraphs of text on a page. And in this case, a group selector does even more by formatting every instance of three different headline tags!

Creating and Applying an ID Selector

ID selectors are used to style a single tag. You create the style and add an ID attribute to a tag on the page, and then the properties from that style are applied to that single tag. You'll frequently use IDs to identify form elements, to create in-page links in a page (see the box on page 48), and to use JavaScript to control elements on a page.

Although many web designers now shy away from ID selectors (you'll learn exactly why on page 108), it's still good to know how to use them.

In this exercise, you'll create a style that controls the look of the text that appears at the very top of the page: "CSS: The Missing Manual." This text is considered the page's logo, and you'll create a special ID style to format it.

1. Return to your text editor and the *selector_basics.html* file.

You'll add a new style below the h1 class style you created before.

2. Click after the previous style's closing bracket (}), hit Enter to create a new line, and then type #logo {.

ID selectors always begin with the pound symbol (#). The style's name indicates that you'll apply this to a page element that's considered the site's logo.

3. Hit Enter again, and then type:

```
font-family: Baskerville, Palatino, sans-serif;
font-size: 2em;
color: rgba(255,255,255,.8);
font-style: italic;
text-align: center;
margin-bottom: 30px;
background-color: rgb(191,91,116);
border-radius: 0 0 10px 10px;
padding: 10px;
```

It looks like a long list of properties, but all it does is set some font properties, background color, and spacing for the logo text.

4. Finish the style by typing the closing brace. The whole thing should look like this:

```
#logo {
  font-family: Baskerville, Palatino, sans-serif;
  font-size: 2em;
  color: rgba(255,255,255,.8);
  font-style: italic;
  text-align: center;
  margin-bottom: 30px;
  background-color: rgb(191,91,116);
  border-radius: 0 0 10px 10px;
  padding: 10px;
}
```

If you save the file and preview it in a web browser, you won't see any difference. That's because this style doesn't *do* anything until you apply it. So you'll add an ID attribute to the page's HTML, indicating where you want the ID style to apply.

5. Find the `<div>` tag near the opening `<body>` tag—it has the text “CSS: The Missing Manual” in it. Add `id="logo"` to the opening div so the HTML looks like this:

```
<div id="logo">
  CSS: The Missing Manual
</div>
```

The `<div>` tag now reflects the formatting defined in the `#logo` style. As with all things CSS, there are many ways to arrive at the same destination: You could instead use a class style and apply it to the `<div>` tag. But in this case you’re using an ID selector, since the point of this style—identifying the logo on the page—is in keeping with the general notion of ID selectors.

6. Save the page, and preview it in a browser.

Now the “CSS: The Missing Manual” text is centered, light-colored, and inside a small box at the top of the page (Figure 3-12).

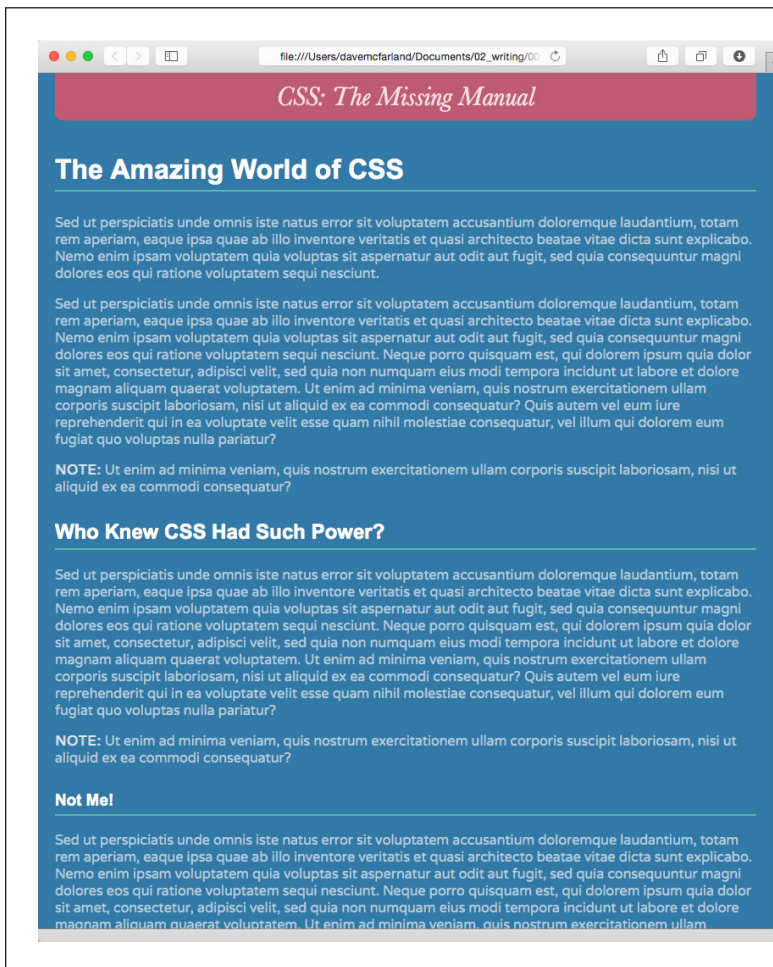


FIGURE 3-12

ID selectors are one way to style unique elements, like the logo bar at the top of this page.

Creating and Applying a Class Selector

Type selectors are quick and efficient, but they're a bit indiscriminate in how they style a page. What if you want to style a single `<p>` tag differently than all the other `<p>` tags on a page? A class selector is the answer.

1. **Return to your text editor and the *selector_basics.html* file.**

Add a new style below the ID selector style you just created.

2. **Click at the end of the closing brace of the `#logo` selector, press Enter, and then type**

```
.note {  
  
}
```

This style's name, `.note`, indicates its purpose: to highlight paragraphs that contain extra bits of information for your site's visitors. Once you create a class style, you can apply it wherever these notes appear—like the third paragraph in this page.

3. **Click in the empty line between the opening `{` and closing `}` and add the following list of properties to the style:**

```
color: black;  
border: 2px solid white;  
background-color: rgb(69,189,102);  
margin-top: 25px;  
margin-bottom: 35px;  
padding: 20px;
```

Notice that you're not using the `rgb()` color values to color the font and border. CSS has several different ways to specify a color, including keywords like `white`, `black`, or `orange`. You'll learn about those on page 147.

If you preview the page now, you see no changes. Like ID selectors, class selectors don't have any effect on a web page until you apply the style in the HTML code.

4. **In the page's HTML, there are two `<p>` tags that begin with the word "Note" inside `<strong>` tags.**

To apply a class style to a tag, simply add a class attribute, followed by the class selector's name—in this case, the `note` style you just created.

5. **Click just after the `p` in the first `<p>` tag, and then type a space followed by `class="note"`. The HTML should now look like this (what you just typed is in bold):**

```
<p class="note"><strong>NOTE:</strong>
```


Be sure *not* to type `class=".note"`. In CSS, the period is necessary to indicate a class style name; in HTML, it's verboten. Repeat this step for the second paragraph (it's just above the `<h3>` tag with the text "Not Me!").

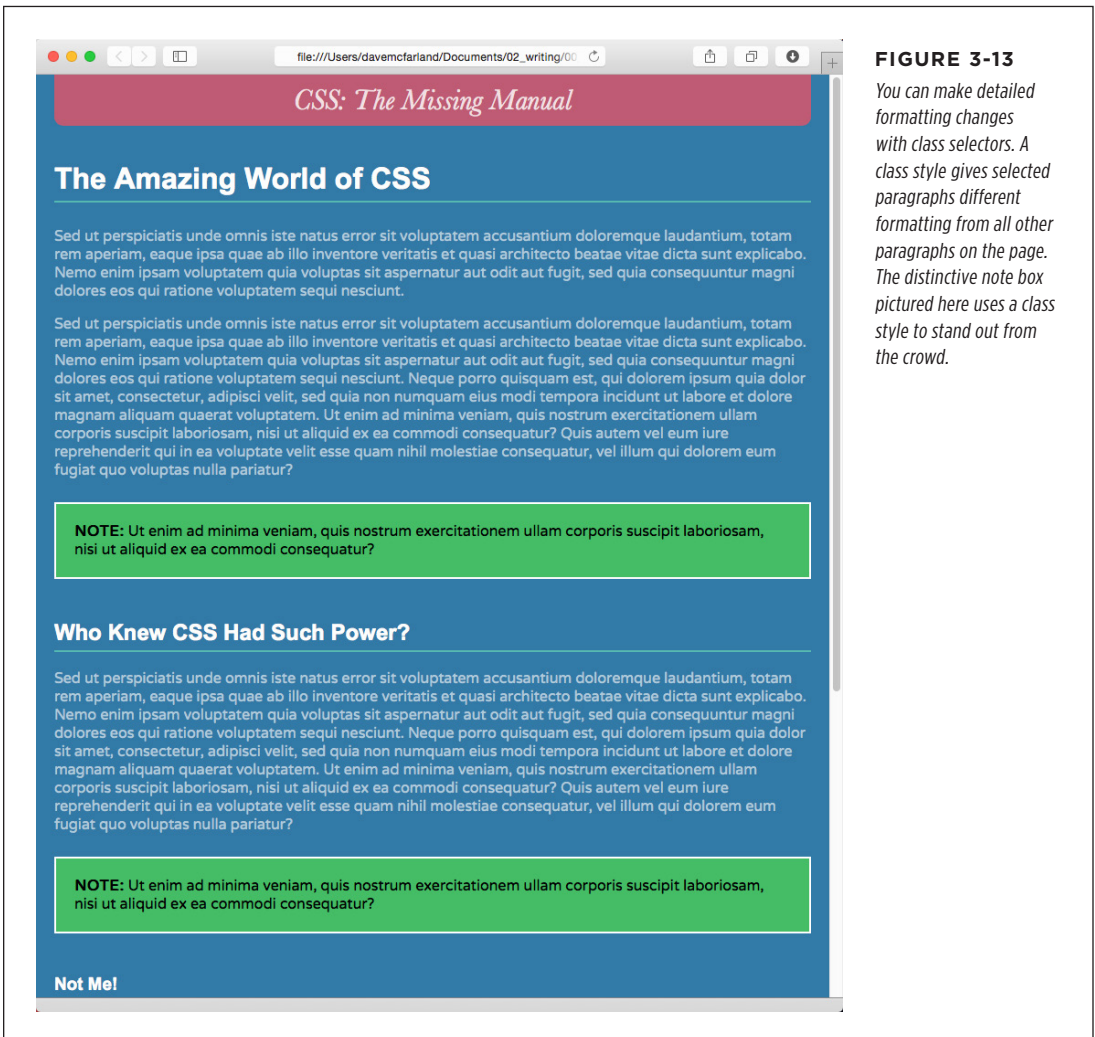
NOTE

There's no reason you can't add this class to other tags as well, not just the `<p>` tag. If you happen to want to apply this formatting to an `<h2>` tag, for example, then your HTML would look like this:

```
<h2 class="note">
```

6. Save and preview the web page in a browser.

The note paragraph is nicely highlighted on the page (see Figure 3-13).

**FIGURE 3-13**

You can make detailed formatting changes with class selectors. A class style gives selected paragraphs different formatting from all other paragraphs on the page. The distinctive note box pictured here uses a class style to stand out from the crowd.

NOTE

If your page doesn't look like Figure 3-13, then you may have mistyped the name of a property or its value. Double-check your code with the steps above. Also, make sure to end each declaration (property: value combination) with a semicolon and conclude the style with a closing brace at the very end. When your style is not working correctly, missing semicolons and closing braces are frequent culprits.

Creating a Descendant Selector

On the [selectors_basics.html](#) page, you applied the `note` class to two paragraphs. Each of those paragraphs begins with the word “Note:” in bold—actually the word is wrapped inside the HTML `<strong>` tag, which all browsers display as bolded text. But what if you want to format those bolded words in bright orange? You could create a tag style for the `<strong>` tag, but that would affect all `<strong>` tags on the page, and you only want to change the strong tag inside those note boxes. One solution would be to create a class style—.noteText, for example—and apply it to each of the `<strong>` tags inside the note boxes. But that's a lot of work, and you might forget to apply the class if you have a lot of pages with those notes.

A better method is to create a descendant selector (page 50), which targets only the `<strong>` tag when it's inside one of these note boxes. Fortunately, that's easy to do.

1. **Return to your text editor and the [selector_basics.html](#) file. Create a new empty line for the descendant selector style.**

If you just completed the previous steps, click after the closing brace of the `.note` style, and then hit Enter.

2. **Type `.note strong {`.**

The last tag in the selector—`strong`—is the element you ultimately want to format. In this case, the style formats the `<strong>` tag only when it's *inside* another tag with the class `note` applied to it. It has no effect on `<strong>` tags inside other paragraphs, lists, or heading tags, for example.

3. **Hit Enter, type `color: #FC6512;`, and then hit Enter again to create another blank line. Finish the style by typing the closing brace character.**

The finished style should look like this:

```
.note strong {  
  color: #FC6512;  
}
```

4. **Save the page and preview it in a web browser.**

The word “Note:” should appear in orange in each of the note boxes on the page.

Descendant selectors are among the most powerful CSS tools. Professional web designers use them extensively to target particular tags without littering the HTML with CSS classes. You'll learn a lot more about descendant selectors as they're used throughout this book.

Finishing Touches

The text on this page expands to fill the browser window. To see this in effect, preview the page and resize your browser window. You'll see the lines of text get wider as you stretch the window. If you have a large monitor, you'll see that the text becomes pretty hard to read past a certain width: The lines of text are too long to read comfortably. Fortunately, you can set a width for the page's content, so it doesn't get too wide to read.

1. Return to your text editor and the *selector_basics.html* file. Create a new empty line for a new style.

If you just completed the previous steps, click after the closing brace of the `.note` strong descendant selector style, and then hit Enter.

2. Add another style:

```
article {  
    max-width: 760px;  
}
```

This is another type selector. It applies to the HTML5 `<article>` tag, which is used to define the content that makes up an article like a blog post, or the content on this page.

The `max-width` property sets the maximum width for the tag, meaning that the article tag will never get wider than 760 pixels. Save the file and preview it in a browser. If you widen the browser window, you'll notice that past 760 pixels, the window gets wider and the blue background of the page appears, but the text no longer expands.

On the other hand, if you make the browser window smaller than 760 pixels, the text lines do get shorter. That's the power of the `max-width` property, which sets a maximum width, but no minimum. This property is extremely useful when you're designing sites that need to work on a variety of screen sizes—desktop computers, laptops, tablets, and smart phones. It's an important part of *responsive design*, which you'll learn about in Chapter 17.

Now that you've limited how far the text can expand, it would be nice to keep the content centered on the screen instead of sticking to the left edge as the browser window expands.

3. Add one more property to the `article` style so it looks like this (addition in bold):

```
article {  
    max-width: 760px;  
    margin: 0 auto;  
}
```

The `margin` property sets the space between an element and other elements around it. You'll read more about margins on page 187, but here this line sets the left and right margins to `auto`, which tells the web browser to automatically figure out how much space to add to the left and right sides of the `article` tag. Once the browser window gets past 760 pixels, the `article` element gets no wider, so the browser basically adds empty space to both the left and right of the tag, in essence centering it in the middle of the browser window.

For fun, you'll add one more advanced style—an adjacent sibling selector discussed on page 67—to format the paragraph immediately following the first headline on the page. (You can achieve the same effect by creating a class style and applying it to that paragraph, but the adjacent sibling selector requires no changes to your HTML.)

4. Add one last style:

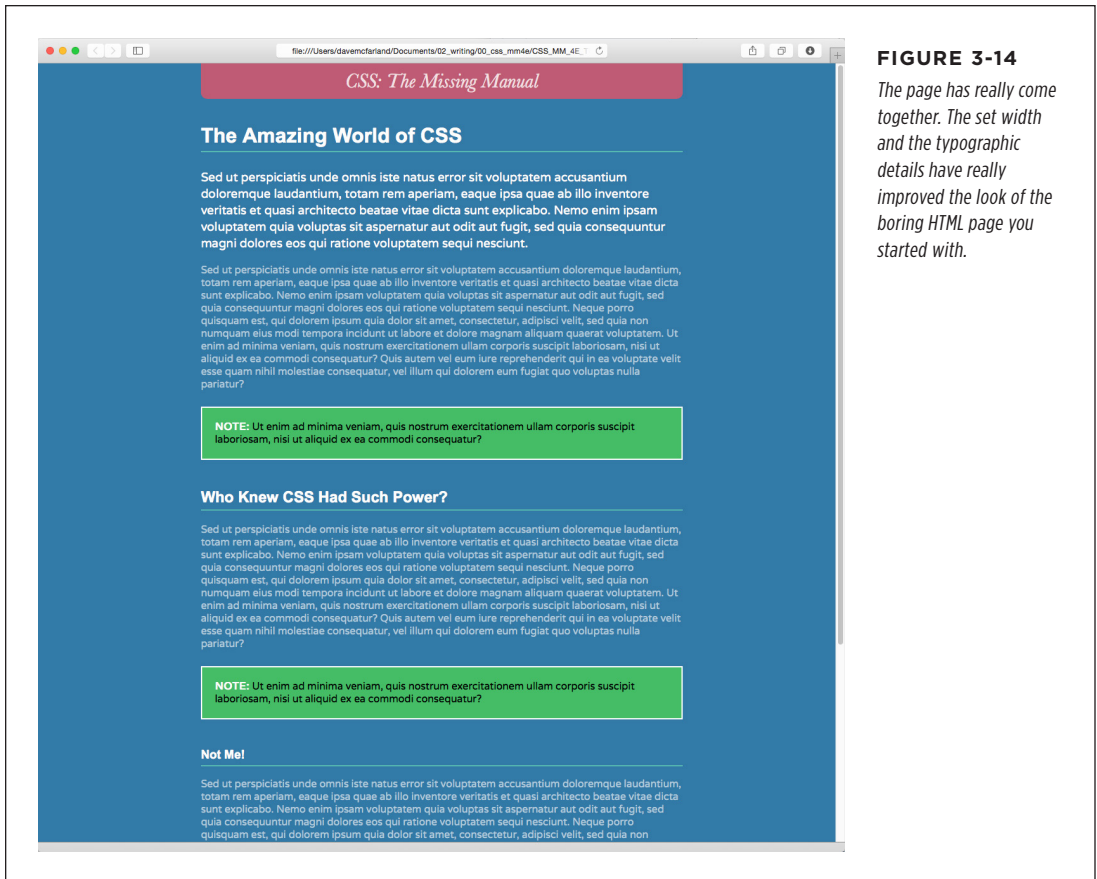
```
h1+p {  
  color: rgb(255,255,255);  
  font-size: 1.2em;  
  line-height: 140%;  
}
```

This style will apply to any paragraph that *immediately* follows an `<h1>` tag—in other words, the first paragraph after the top headline on the page. It won't apply to the second or any subsequent paragraphs. This selector provides an easy way to create a unique look for an introductory paragraph to set it off visually and highlight the beginning of an article.

The style changes the font color and size. The `line-height` property (which you'll read about on page 163) controls the space between lines in a paragraph (also known as *leading*).

If you preview the page now, you'll see that the top paragraph is white and its text is larger, and there's more space between each line of text (see Figure 3-14). If you actually deleted this paragraph in the HTML, you'd see that the remaining paragraph would suddenly be white with larger text, since it would be the new adjacent sibling of the `<h1>` tag.

And there you have it: a quick tour through various selector types. You'll get familiar with all of these selectors (and more) as you go through the tutorials later in the book, but by now, you should be getting the hang of the different types and why you'd use one over the other.

**FIGURE 3-14**

The page has really come together. The set width and the typographic details have really improved the look of the boring HTML page you started with.